

 Tip

Sentence Transformers v5.5 recently released, introducing the [train-sentence-transformers](#) Agent Skill. Using an AI coding agent (Claude Code, Codex, Cursor, Gemini CLI, ...)? Install it via `hf skills add train-sentence-transformers [--global] [--claude]` and ask your agent to train or finetune an embedding, reranker, or sparse encoder model on your data. See the [v5.5.0 Release Notes](#) for more details.

## SentenceTransformers Documentation

Sentence Transformers (a.k.a. SBERT) is the go-to Python module for using and training state-of-the-art embedding and reranker models. It can be used to compute embeddings from text, images, audio, or video using Sentence Transformer models ([quickstart](#)), to calculate similarity scores using Cross-Encoder (a.k.a. reranker) models ([quickstart](#)), or to generate sparse embeddings using Sparse Encoder models ([quickstart](#)). This unlocks a wide range of applications, including [semantic search](#), [semantic textual similarity](#), and [paraphrase mining](#).

A wide selection of over [10,000 pre-trained Sentence Transformers models](#) are available for immediate use on  Hugging Face, including many of the state-of-the-art models from the [Massive Text Embeddings Benchmark \(MTEB\) leaderboard](#). Additionally, it is easy to train or finetune your own [embedding models](#), [reranker models](#), or [sparse encoder models](#) using Sentence Transformers, enabling you to create custom models for your specific use cases.

Sentence Transformers was created by [UKP Lab](#) and is being maintained by  [Hugging Face](#). Don't hesitate to open an issue on the [Sentence Transformers repository](#) if something is broken or if you have further questions.

## Usage

 See also

See the [Quickstart](#) for more quick information on how to use Sentence Transformers.

Working with Sentence Transformer models is straightforward:

## Sparse Encoder Models

### Text

### Multimodal

```
from sentence_transformers import SentenceTransformer

# 1. Load a pretrained Sentence Transformer model
model = SentenceTransformer("sentence-
transformers/all-MiniLM-L6-v2")

# The sentences to encode
sentences = [
    "The weather is lovely today.",
    "It's so sunny outside!",
    "He drove to the stadium.",
]

# 2. Calculate embeddings by calling model.encode()
embeddings = model.encode(sentences)
print(embeddings.shape)
# [3, 384]

# 3. Calculate the embedding similarities
similarities = model.similarity(embeddings,
embeddings)
print(similarities)
# tensor([[1.0000, 0.6660, 0.1046],
#         [0.6660, 1.0000, 0.1411],
#         [0.1046, 0.1411, 1.0000]])
```

You can install *sentence-transformers* using pip:

```
pip install -U sentence-
transformers
```

We recommend **Python 3.10+** and **PyTorch 1.11.0+**. See [installation](#) for further installation options.

## What Next?

Consider reading one of the following sections to answer the related questions:

- **Embedding Models:**
  - How to **use** Sentence Transformer models? [Sentence Transformers > Usage](#)
  - What Sentence Transformer **models** can I use? [Sentence Transformers > Pretrained Models](#)
  - How do I make Sentence Transformer models **faster**? [Sentence Transformers > Usage > Speeding up Inference](#)
  - How do I **train/finetune** a Sentence Transformer model? [Sentence Transformers > Training Overview](#)

- **Reranker Models:**

- How to **use** Cross Encoder models? [Cross Encoder > Usage](#)
- What Cross Encoder **models** can I use? [Cross Encoder > Pretrained Models](#)
- How do I make Cross Encoder models **faster**? [Cross Encoder > Usage > Speeding up Inference](#)
- How do I **train/finetune** a Cross Encoder model? [Cross Encoder > Training Overview](#)
- **Sparse Encoder Models:**
  - How to **use** Sparse Encoder models? [Sparse Encoder > Usage](#)
  - What Sparse Encoder **models** can I use? [Sparse Encoder > Pretrained Models](#)
  - How do I make Sparse Encoder models **faster**? [Sparse Encoder > Usage > Speeding up Inference](#)
  - How do I **train/finetune** a Sparse Encoder model? [Sparse Encoder > Training Overview](#)
  - How do I **integrate** Sparse Encoder models with search engines? [Sparse Encoder > Vector Database Integration](#)

## Companion Blog Posts

The following Hugging Face blog posts complement this documentation with narrative walkthroughs and full training examples:

- Training guides:
  - [Training and Finetuning Embedding Models](#): end-to-end training of bi-encoder embedding models.
  - [Training and Finetuning Reranker Models](#): training Cross Encoder (reranker) models.
  - [Training and Finetuning Sparse Embedding Models](#): training SPLADE and other sparse encoders.
- Multimodal:
  - [Multimodal Embedding & Reranker Models](#): text, image, audio, and video models through a single API.
  - [Training and Finetuning Multimodal Embedding & Reranker Models](#): finetuning a multimodal embedding model for Visual Document Retrieval.
- Efficiency techniques:
  - [Introduction to Matryoshka Embedding Models](#): variable-size embeddings that truncate gracefully.
  - [Train 400x faster Static Embedding Models](#): attention-free CPU-friendly embedding models.
  - [Binary and Scalar Embedding Quantization for Significantly Faster & Cheaper Retrieval](#): post-training compression of embedding vectors.

# Citing

If you find this repository helpful, feel free to cite our publication [Sentence-BERT: Sentence Embeddings using Siamese BERT-Networks](#):

```
@inproceedings{reimers-2019-sentence-bert,  
  title = "Sentence-BERT: Sentence Embeddings using Siamese BERT-Networks",  
  author = "Reimers, Nils and Gurevych, Iryna",  
  booktitle = "Proceedings of the 2019 Conference on Empirical Methods in Natural Language Processing",  
  month = "11",  
  year = "2019",  
  publisher = "Association for Computational Linguistics",  
  url = "https://arxiv.org/abs/1908.10084",  
}
```

If you use one of the multilingual models, feel free to cite our publication [Making Monolingual Sentence Embeddings Multilingual using Knowledge Distillation](#):

```
@inproceedings{reimers-2020-multilingual-sentence-bert,  
  title = "Making Monolingual Sentence Embeddings Multilingual using Knowledge Distillation",  
  author = "Reimers, Nils and Gurevych, Iryna",  
  booktitle = "Proceedings of the 2020 Conference on Empirical Methods in Natural Language Processing",  
  month = "11",  
  year = "2020",  
  publisher = "Association for Computational Linguistics",  
  url = "https://arxiv.org/abs/2004.09813",  
}
```

If you use the code for [data augmentation](#), feel free to cite our publication [Augmented SBERT: Data Augmentation Method for Improving Bi-Encoders for Pairwise Sentence Scoring Tasks](#):

```
@inproceedings{thakur-2020-AugSBERT,  
  title = "Augmented {SBERT}: Data Augmentation Method for Improving Bi-Encoders for Pairwise  
Sentence Scoring Tasks",  
  author = "Thakur, Nandan and Reimers, Nils and Daxenberger, Johannes and Gurevych, Iryna",  
  booktitle = "Proceedings of the 2021 Conference of the North American Chapter of the  
Association for Computational Linguistics: Human Language Technologies",  
  month = jun,  
  year = "2021",  
  address = "Online",  
  publisher = "Association for Computational Linguistics",  
  url = "https://www.aclweb.org/anthology/2021.naacl-main.28",  
  pages = "296--310",  
}
```